

阶段 1 学习总结（2026-05-27，4h）

本阶段交付物

文件	说明
docs/REQUIREMENTS.md	需求规格说明书（功能需求 + 非功能需求 + 里程碑规划 + 硬件资源表）
README.md	项目自述文件
.gitignore	Git 忽略规则（内核产物、编译目标、编辑器临时文件）
Makefile	顶层构建文件（KERNEL_DIR / ARCH / CROSS_COMPILE 可配置）
driver/Makefile	驱动层 Kbuild Makefile（obj-m := hello.o）
driver/hello.c	骨架内核模块（验证交叉编译工具链）
app/Makefile	用户层编译框架
scripts/build.sh	一键编译脚本
dts/.gitkeep	设备树目录占位

新接触的知识点

知识点	说明	涉及文件
项目目录结构规范化	嵌入式 Linux 项目的标准分层目录：driver / app / dts / docs / scripts，各层职责明确	整个项目骨架
需求规格说明书	在写代码之前先明确功能需求、非功能需求、硬件资源规划，形成可追溯的文档基线	docs/REQUIREMENTS.md
Kbuild 两阶段编译	用户空间 <code>make -C \$(KERNEL_DIR) M=\$(CURDIR) modules</code> 调用内核构建系统，内核 Makefile 读取本地 <code>obj-m</code> 变量决定编译哪些模块	driver/Makefile:12
<code>?=</code> 条件赋值	Makefile 中 <code>?=</code> 只在变量未设置时才赋值，允许命令行覆盖默认值，是嵌入式项目 Makefile 的标准写法	Makefile:2-4
<code>module_init</code> / <code>module_exit</code>	内核模块的入口和出口宏，展开为 <code>__attribute__((section(".init.text"))) 等链接器指令</code>	driver/hello.c:14,19
<code>pr_info</code>	内核打印宏，等同 <code>printk(KERN_INFO ...)</code> ，比 <code>printk</code> 更简洁	driver/hello.c:9,14
Conventional Commits	<code>feat:</code> / <code>fix:</code> / <code>docs:</code> 等前缀的提交信息规范，便于后续生成 CHANGELOG	Git 提交记录
CRLF 警告	Windows 下 Git 默认将 LF 转为 CRLF，而 Linux 内核源码要求 LF。可通过 <code>.gitattributes</code> 或 <code>git config</code>	本次提交时的 warning

知识点	说明	涉及文件
	<code>core.autocrlf</code> 控制	

踩过的坑

问题	原因	解决方案
dts 空目录无法被 git 跟踪	Git 只跟踪文件，不跟踪空目录	在 <code>dts/</code> 下放 <code>.gitkeep</code> 占位文件
CRLF 警告	Windows 的 Git 默认 <code>core.autocrlf=true</code>	警告不影响功能；后续如有需要可在 <code>.gitattributes</code> 中为 <code>.c/.h/.S</code> 指定 <code>text eol=lf</code>

代码规范要点

- Makefile 变量使用 `?=` 条件赋值，让用户可通过命令行或环境变量覆盖
- Kbuild 中 `obj-m` 列出要编译的模块目标文件（`.o`），内核构建系统会自动寻找对应的 `.c` 源文件
- `.gitignore` 应覆盖：内核编译产物（`.ko/.mod.c/.mod.o`）、用户程序二进制、设备树编译产物（`.dtb/.dtbo`）、编辑器临时文件
- 提交信息使用 Conventional Commits 格式：`feat:` 前缀表示新功能

下一阶段预告

阶段 2：设备树设计 — 从硬件原理图出发，规范化设计设备树节点（GPIO / I2C / SPI / UART），编写 `dts/imx6ull-smart-monitor.dtsi`，并用 `dtc` 做语法验证。